

Universidad Autónoma de Querétaro
Facultad de Informática

Proyecto Final
Inteligencia Artificial

Clasificación de Cartas de Póker

Alan Espinosa Torres

Ángel Gabriel López Molina

Víctor Hugo Ramírez Moreno

Francisco Luna Fernández

Víctor Hugo Ruiz Alcántara

Ricardo Rojas Álvarez

Objetivo

Utilizando el dataset de cartas de póker (*Cards Image Dataset-Classification*, 2022), elegimos por generar un modelo de clasificación de 53 clases, las cuales corresponden a todas las cartas de una baraja de póker estándar, elegimos este dataset por su gran cantidad de clases y gracias a nuestra disponibilidad de equipo de computo nos decidimos por usar las imágenes directas del dataset de 224x224 pixeles.

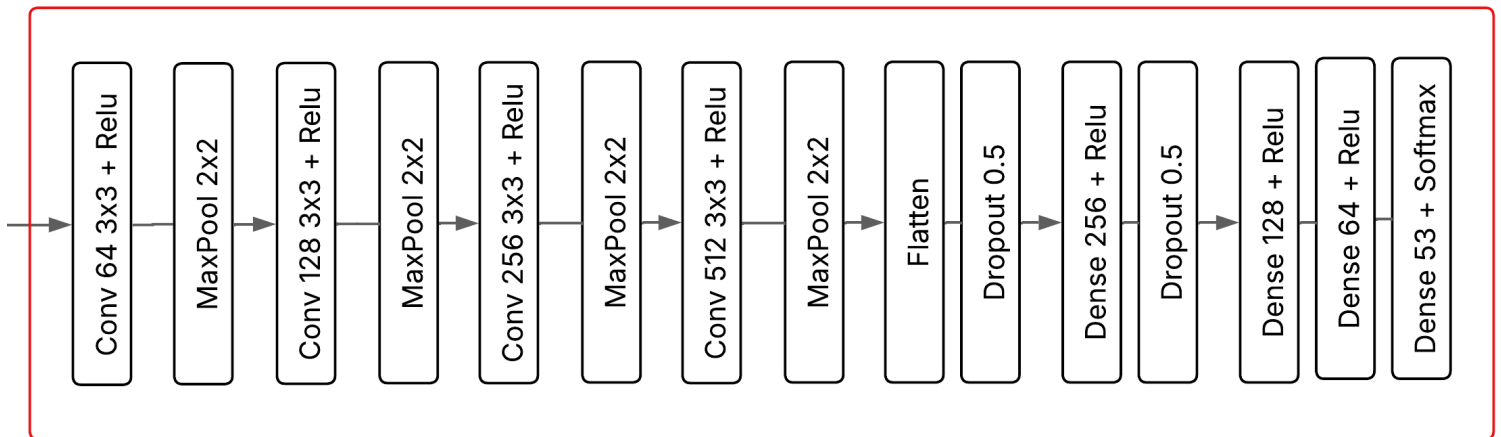
Dataset

El dataset (*Cards Image Dataset-Classification*, 2022), cuenta con 53 clases, cada una correspondiente a cada carta de un abaraja de póker, cada clase cuenta con un mínimo de 108 fotografías de 224x224 pixeles y un máximo de 138.

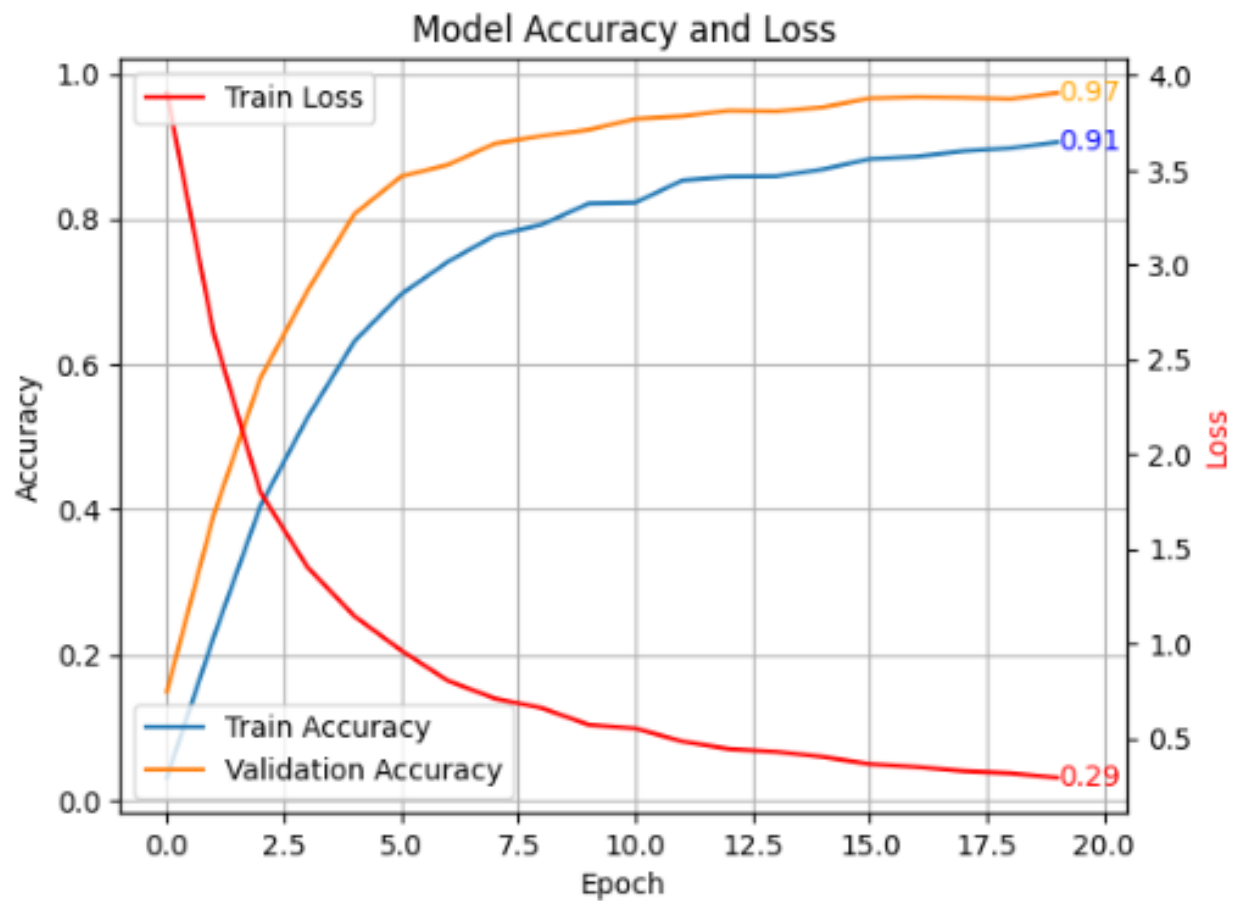
Arquitectura

Después de varias iteraciones se decidió usar una variación propia en la arquitectura VGG-16 la cual obtuvo el mejor rendimiento, teniendo la siguiente estructura:

Modelado



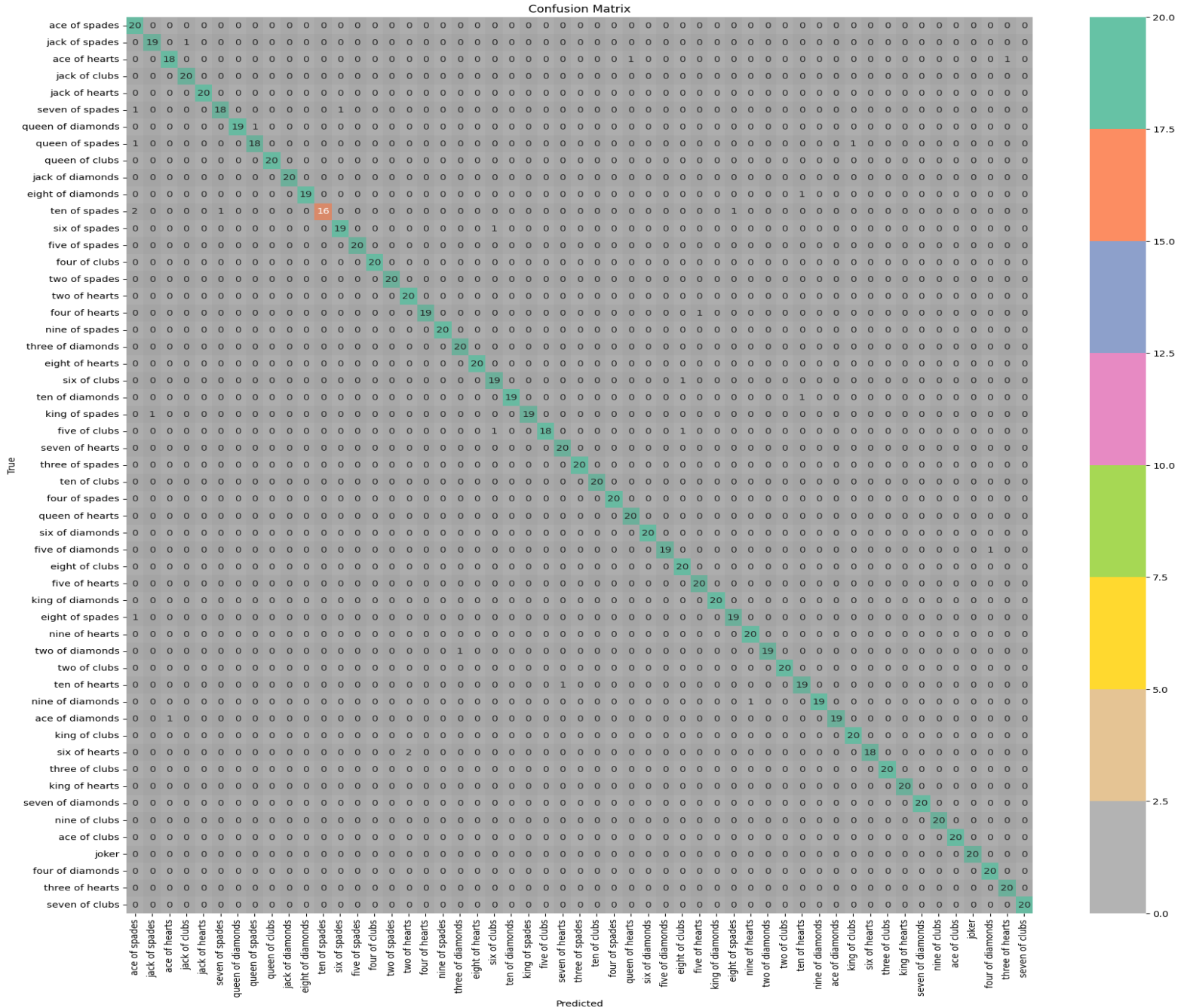
Este modelo consiguió un rendimiento del 97% en 20 épocas con un *loss* de 0.29 en entrenamiento, representado por la siguiente gráfica:



En el testeo del modelo se logró un rendimiento del 98.49% y un *loss* del 0.07:

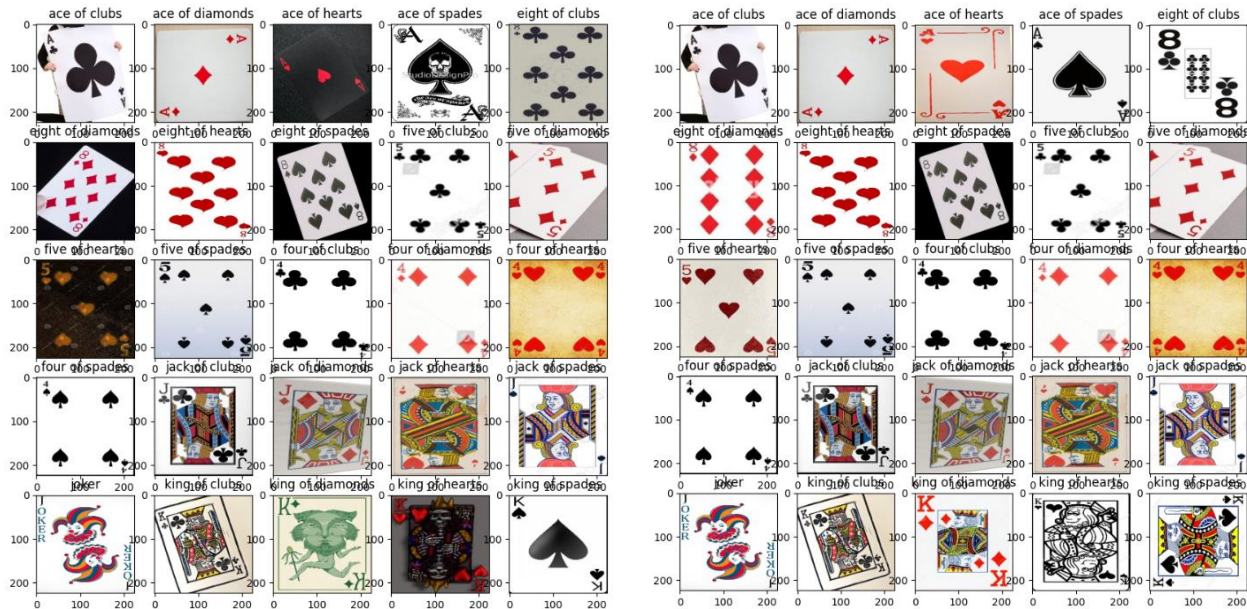
```
34/34 [=====] - 2s 46ms/step - loss: 0.0764 - accuracy: 0.9849
```

La matriz de confusión generada por este modelo se muestra a continuación, donde es importante resaltar que incluso para las clases con peor rendimiento, las predicciones eran del mismo palo de la carta demostrando la capacidad del modelo.



Técnicas de mejora

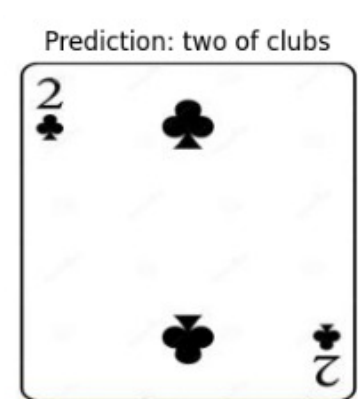
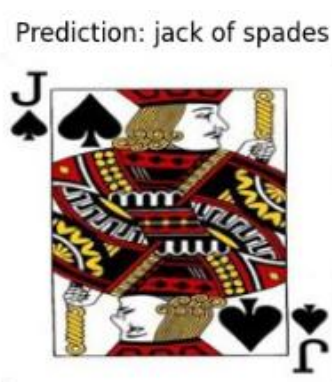
El dataset original (*Cards Image Dataset-Classification*, 2022), contenía imágenes de barajas estilizadas (imagen a la izquierda), las cuales para nuestro caso de uso representaban unos edge cases que introducían un sesgo importante que solo permitía un $\sim 70\%$ de rendimiento.



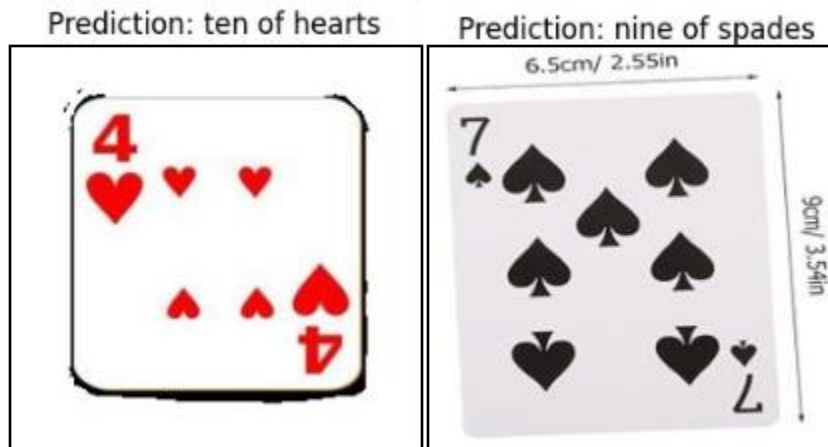
Nos decidimos por limpiar el dataset de todos estos edge cases (imagen a la derecha), dejando por lo menos 100 imágenes por clase para el entrenamiento.

Resultados

Como pruebas preliminares se hizo la predicción de cartas encontradas en internet, de las cuales el modelo si pudo reconocer de manera correcta.

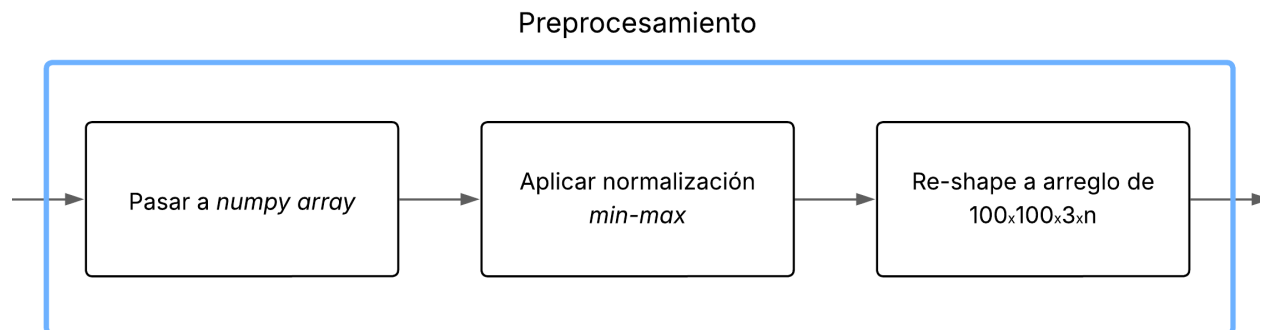


Aunque una de las limitaciones encontradas fue que las imágenes del dataset utilizado están recortadas de tal manera que solo la carta es visible, por lo que al introducir imágenes que no rellenan completamente sus bordes el modelo no hace una predicción correcta, a pesar de esto el palo de cada carta si fue el correcto:

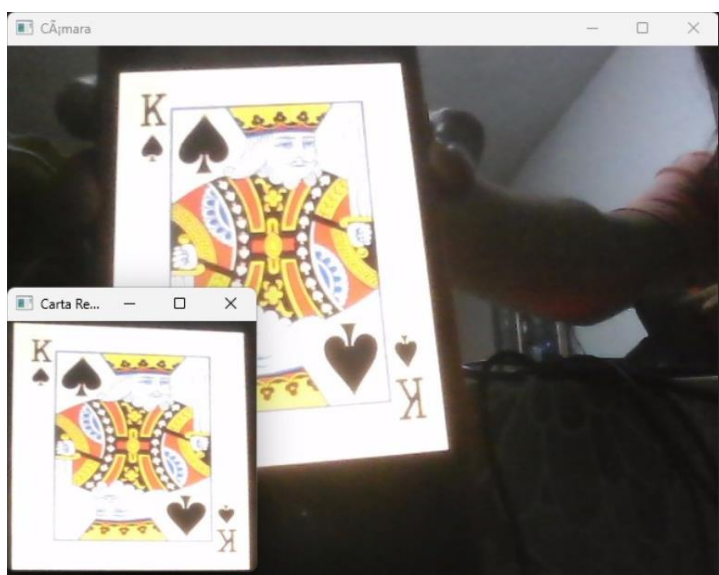
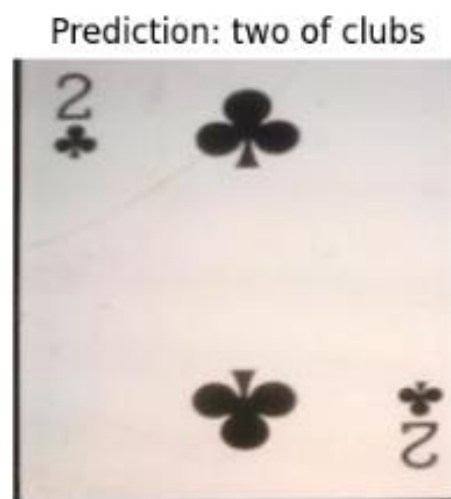
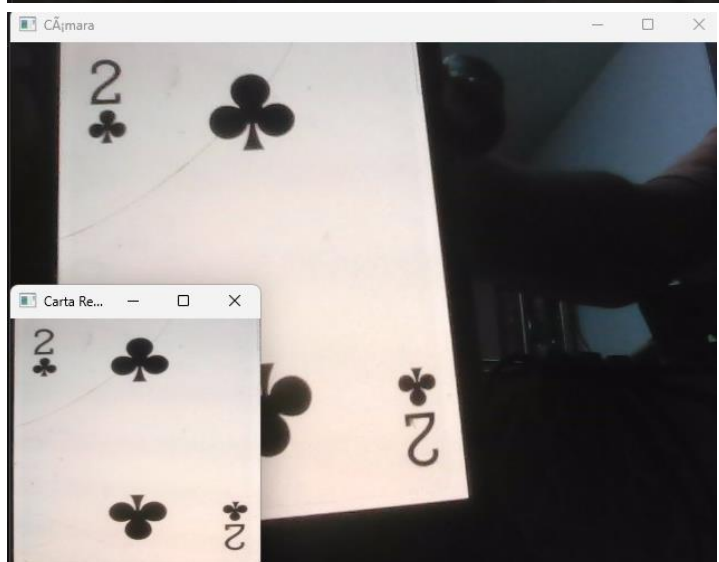
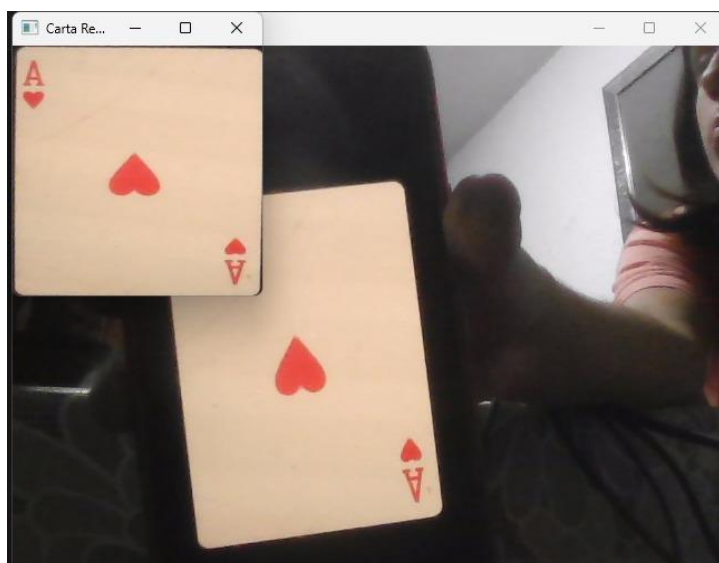


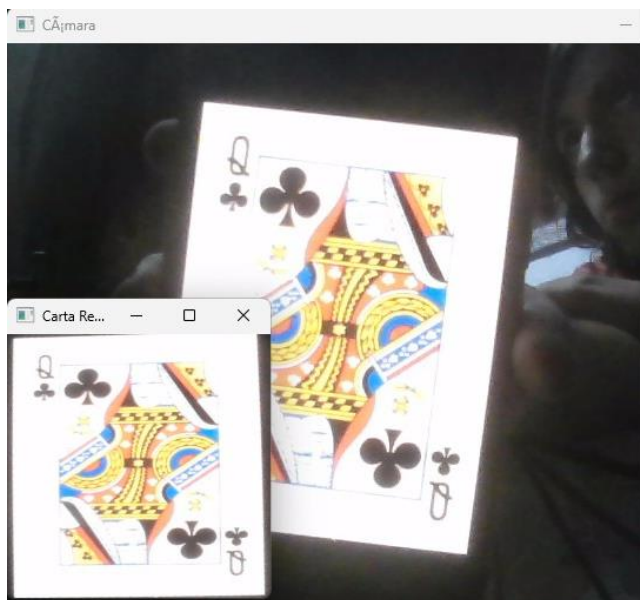
Implementación tiempo real

Con un simple programa de Python y utilizando openCV se hace la detección de un rectángulo en la imagen, recortarlo y rescalarlo a las mismas dimensiones de 224x224 para después hacer los mismos pasos del preprocesamiento y hacer la predicción con el modelo pre calculado:

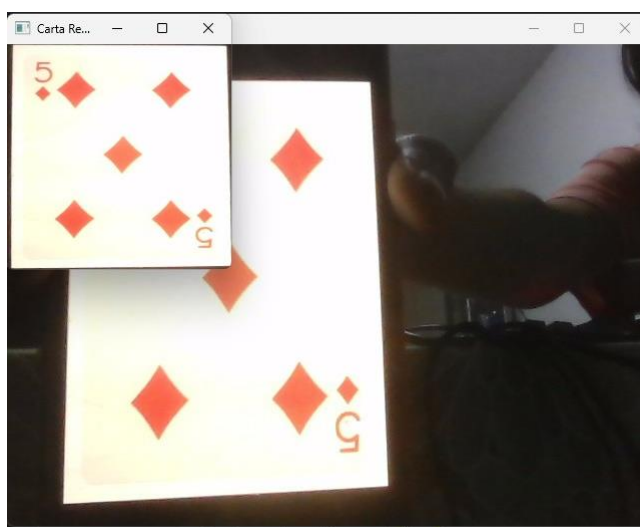


Aquí se muestra el programa en tiempo real donde se detecta la presencia de un rectángulo en la imagen y la imagen recortada y re-escalada con la predicción del modelo:





Prediction: queen of clubs



Prediction: five of diamonds



Conclusión

El modelo que generamos tuvo un rendimiento muy superior al esperado, e incluso superior a algunos modelos presentados por otras personas en el propio dataset, principalmente por la limpieza de datos que realizamos, la cual dio un ~27% de mejora en el rendimiento.

La arquitectura se basó en la VGG-16, la cual después de varias iteraciones agregando y quitando capas se consiguió un rendimiento del 98.49%.

Impresionantemente al subir el brillo de nuestra prueba y presentar una imagen saturada el modelo es aún capaz de clasificarla de manera correcta.

Referencias

Cards Image Dataset-Classification. (2022, 17 noviembre). Kaggle.
<https://www.kaggle.com/datasets/gpiosenka/cards-image-datasetclassification/data?select=train>

Guardar y cargar modelos. (s. f.). TensorFlow.
https://www.tensorflow.org/tutorials/keras/save_and_load?hl=es-419